Clase 04 - Funciones: de algoritmo a código reutilizable

Week 01 - Logic | Primer ejercicio de class_04_functions

Ruta actual: Python_Engineering / 01_python_bootcamp / week_01_logic / class_04_functions

Dónde estamos

Las tres clases anteriores ya están cerradas: ecuación -> algoritmo, condicionales y loops. Ahora seguimos dentro de week_01_logic, pero pasamos a una nueva clase: class_04_functions. El objetivo ya no es aprender una sintaxis nueva por sí sola, sino empezar a transformar scripts lineales en piezas reutilizables.

Bloque	Estado
Class 01 - Equation to Algorithm	Completada
Class 02 - Conditionals	Completada
Class 03 - Loops	Completada
Class 04 - Functions	AHORA

Objetivos de esta clase

Al terminar esta clase deberías poder explicar y usar correctamente: def, parámetros, argumentos, return, scope local, llamadas a funciones y reutilización de código. También empezaremos a separar cálculo, validación y presentación.

Idea clave

Una función es una pequeña máquina: recibe datos, ejecuta una tarea concreta y, si procede, devuelve un resultado.

```
INPUTS  ->  FUNCTION  ->  OUTPUT
              |
              +-- internal logic
```

1. Qué es una función

Hasta ahora hemos escrito instrucciones una detrás de otra. Si quisiéramos calcular presión dinámica diez veces, repetiríamos código. Una función nos permite encapsular ese cálculo una sola vez y reutilizarlo.

```
def calculate_dynamic_pressure(rho, speed):
    dynamic_pressure = 0.5 * rho * speed**2
    return dynamic_pressure
```

La palabra def crea la función. rho y speed son parámetros. return devuelve el resultado al lugar desde el que llamamos a la función.

Parámetros vs argumentos

Concepto	Ejemplo	Significado
Parámetro	rho, speed	Nombres usados al definir la función
Argumento	1.225, 100.0	Valores reales usados al llamar a la función

```
q = calculate_dynamic_pressure(1.225, 100.0)
```

Aquí, 1.225 y 100.0 son argumentos. Dentro de la función, Python los asigna a rho y speed respectivamente.

2. return no es print

Esta diferencia es fundamental. print() muestra información al usuario. return entrega un valor para que el resto del programa pueda seguir trabajando con él.

```
def bad_example(rho, speed):
    q = 0.5 * rho * speed**2
    print(q)

def useful_example(rho, speed):
    q = 0.5 * rho * speed**2
    return q
```

Con la segunda versión puedes hacer después q = useful_example(...), comparar q, almacenarlo, escribirlo en un archivo o utilizarlo en otra ecuación.

3. Scope: variables locales

Una variable creada dentro de una función normalmente es local: existe durante esa llamada y no tiene por qué existir fuera de la función.

```
def calculate_dynamic_pressure(rho, speed):
    q = 0.5 * rho * speed**2
    return q

result = calculate_dynamic_pressure(1.225, 100.0)
# q es local; result existe fuera
```

Esta separación reduce errores y hace que cada función tenga una responsabilidad concreta.

4. Ejercicio 09 - Dynamic Pressure Function

Archivo: exercises/ex09_dynamic_pressure_function.py

Queremos reutilizar el primer ejercicio del curso, pero ahora convirtiéndolo en una función.

Modelo físico

q = 0.5 * rho * V^2

Requisitos del ejercicio

- Crear una función llamada calculate_dynamic_pressure.
- La función debe recibir rho y speed como parámetros.
- Debe validar que rho >= 0 y speed >= 0.
- Debe calcular la presión dinámica.
- Debe devolver el resultado con return.
- El print final debe estar fuera de la función.
- Probar la misma función con varios pares de valores sin copiar la ecuación.

Importante: no quiero todavía clases, módulos ni input() del usuario. El foco es entender una función.

5. Diseña antes de programar

```
FUNCTION
--------
Name: calculate_dynamic_pressure

INPUTS
------
rho
speed

VALIDATION
----------
rho >= 0
speed >= 0

CALCULATION
-----------
q = 0.5 * rho * speed**2

OUTPUT
------
return q
```

No copies una solución completa. Utiliza este diseño para escribirla tú. La meta es que puedas explicar qué entra, qué ocurre dentro y qué sale.

6. Casos de prueba

rho [kg/m3]	speed [m/s]	Resultado esperado
1.225	100	6125 Pa
1.225	0	0 Pa
0	100	0 Pa
-1	100	Rechazar
1.225	-20	Rechazar

Prueba también que puedes llamar a la función más de una vez:

```
q1 = calculate_dynamic_pressure(...)
q2 = calculate_dynamic_pressure(...)
q3 = calculate_dynamic_pressure(...)
```

Si para cambiar de caso solo modificas los argumentos, has conseguido la primera forma real de reutilización de código.

7. Qué observar con el debugger

Pon un breakpoint en la primera línea de la función. Observa cómo Python entra en ella, crea los parámetros locales, calcula el resultado, ejecuta return y vuelve a la línea desde la que fue llamada.

```
caller -> function -> return -> caller
```

8. Bonus opcional

Si terminas rápido, prueba a crear una segunda función exclusivamente para validar los inputs. No es obligatorio todavía; servirá para empezar a pensar en single responsibility: una función, una tarea clara.

No avances todavía a funciones anidadas, lambdas o decoradores. Primero queremos dominar bien las bases.

Learning Tracker

Concepto	Estado tras EX09
def	Introducido
Parámetros	Introducido
Argumentos	Introducido
return	Introducido
print vs return	Introducido
Scope local	Introducido
Reutilización	Práctica inicial
Validación dentro de función	Práctica inicial

Siguiente paso previsto

Cuando cierres EX09, seguiremos dentro de week_01_logic -> class_04_functions. El siguiente ejercicio reutilizará la Rocket Equation para practicar múltiples parámetros, validación y retorno. Te avisaré explícitamente cuando cambiemos de clase o de week.